

ReactJS Hands-On - Context API (Employee Management App)

1. Objective

Convert the theme name in the employee management app from being passed down through props (App -> EmployeeList -> EmployeeCard) to being shared through the React Context API.

2. Prerequisites

Node.js, NPM, Visual Studio Code

3. Restore Node Modules

Command typed in the VS Code terminal after unzipping the application:

```
npm install
```

4. Run the Application (Before Conversion)

Command used to view the original props-based output before making changes:

```
npm start
```

5. Original Structure (Before) - Theme Passed as Props

Before conversion, App.js held the theme in state and passed it down as a prop to EmployeeList, which passed it further down to EmployeeCard:

```
// App.js (before)
<EmployeeList theme={theme} />

// EmployeeList.js (before)
<EmployeeCard employee={emp} theme={theme} />

// EmployeeCard.js (before)
function EmployeeCard({ employee, theme }) {
  return (
    <div className={`card ${theme}`}>
      <h4>{employee.name}</h4>
      <button className={`btn-${theme}`}>View Profile</button>
    </div>
  );
}
```

6. ThemeContext.js - Creating the Context

```
import { createContext } from 'react';

const ThemeContext = createContext('light');

export default ThemeContext;
```

7. App.js - Using ThemeContext.Provider

The ThemeContext is imported, the entire JSX is wrapped in ThemeContext.Provider, the value comes from state, and EmployeeList is no longer passed the theme prop:

```
import React, { useState } from 'react';
import ThemeContext from './ThemeContext';
import EmployeeList from './EmployeeList';

function App() {
  const [theme, setTheme] = useState('light');

  return (
    <ThemeContext.Provider value={theme}>
      <div className={theme}>
        <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
          Toggle Theme
        </button>
        <EmployeeList />
      </div>
    </ThemeContext.Provider>
  );
}

export default App;
```

8. EmployeeList.js - No Longer Passing Theme as Prop

```
import React from 'react';
import EmployeeCard from './EmployeeCard';

const employees = [
  { id: 1, name: 'Aditi Sharma', role: 'Software Engineer' },
  { id: 2, name: 'Rohan Mehta', role: 'QA Analyst' },
  { id: 3, name: 'Kavya Iyer', role: 'Project Manager' }
];

function EmployeeList() {
  return (
    <div>
      {employees.map(emp => (
        <EmployeeCard key={emp.id} employee={emp} />
      ))}
    </div>
  );
}

export default EmployeeList;
```

9. EmployeeCard.js - Reading Theme with useContext()

```
import React, { useContext } from 'react';
import ThemeContext from './ThemeContext';

function EmployeeCard({ employee }) {
  const theme = useContext(ThemeContext);
```

```

    return (
      <div className={`card ${theme}`}>
        <h4>{employee.name}</h4>
        <p>{employee.role}</p>
        <button className={`btn-${theme}`}>View Profile</button>
      </div>
    );
  }
}

export default EmployeeCard;

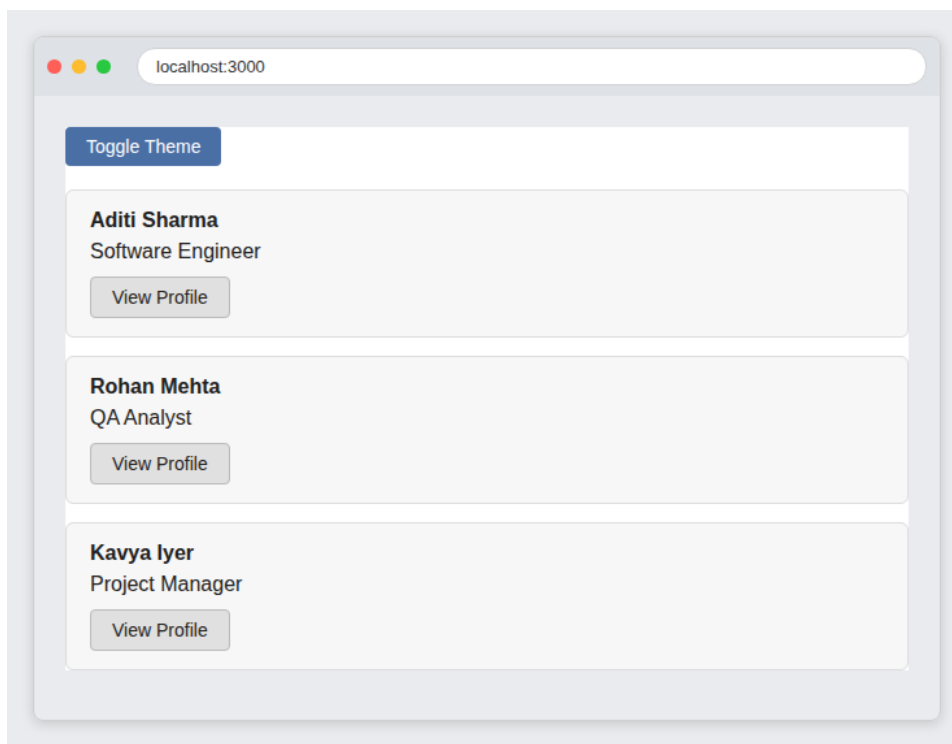
```

10. Run the Application (After Conversion)

```
npm start
```

11. Output - localhost:3000

Browser output after the conversion. All 3 employee cards receive the theme value through Context, with no theme prop passed through EmployeeList:



12. Observation

Earlier, the theme name had to be passed as a prop through every intermediate component (App -> EmployeeList -> EmployeeCard), even though EmployeeList itself never used it. After the conversion, ThemeContext.Provider in App.js supplies the theme value directly, and EmployeeCard reads it with useContext(ThemeContext) without EmployeeList needing to know about it at all. This removes the prop-drilling and matches the React architect's recommendation.